Beta Function Calculator

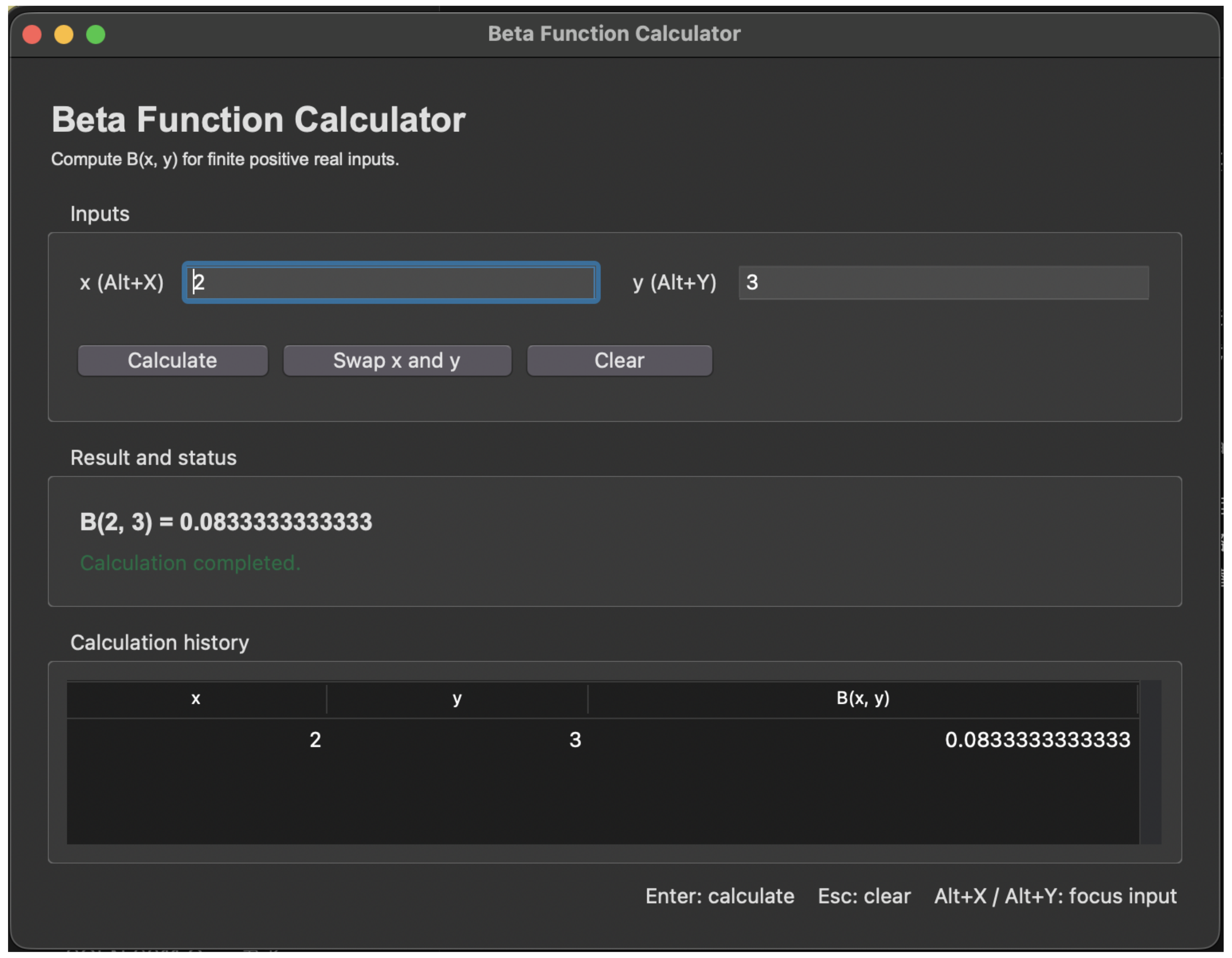
Deliverable 3: Numerical Correctness, Code Quality, Accessibility, and Testing

Wei Huang
SOEN 6011 | Summer 2026

Semantic Version 1.0.0

$$B(x,y)=\frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)},\qquad x>0,\ y>0$$

Final Graphical Interface



The live interface demonstrates a known value, visible calculation status, and persistent history. Native Tkinter widgets preserve familiar focus and tab behavior.

Accessibility and Recovery

Keyboard access Enter, Esc, Alt+X, Alt+Y	Recognition over recall visible labels and actions
Helpful recovery field-specific text messages	Status is not color-only words identify success or error

The GUI aims to be accessible through readable labels, visible focus, redundant text feedback, keyboard operation, and recoverable validation. These choices align the interface with the user's task model (Kamthan, 2026c).

Exception policy: expected input and numeric-range errors receive specific messages; unexpected arithmetic failures are not hidden behind a generic message and remain available for diagnosis.

D3 conclusion: the release improves numerical stability, accessibility, and software quality through a stable log-beta implementation, explicit range behavior, and repeatable verification.

Problem 7: Style and Static Analysis

Code quality evidence

```
$ flake8 beta_core.py beta_function_gui.py test_beta_core.py debug_beta.py verify_accuracy.py
No style violations found (exit status 0).

$ pylint beta_core.py beta_function_gui.py test_beta_core.py debug_beta.py verify_accuracy.py
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Flake8 reports no PEP 8 style violations; Pylint rates the final source **10.00/10**. Both snapshots were generated from version 1.0.0.

Problem 7: Debugger Evidence

pdb debugger evidence

```
$ python3 -m pdb debug_beta.py
(Pdb) Breakpoint 1 at beta_core.py:256
(Pdb) > beta_core.py(258)beta_function()
-> if not is_finite(x_value):
(Pdb) (2.0, 3.0)
(Pdb) > beta_core.py(260)beta_function()
-> if not is_finite(y_value):
(Pdb) > beta_core.py(262)beta_function()
(Pdb) > beta_core.py(264)beta_function()
(Pdb) B(2, 3) = 0.08333333333333313
```

A reproducible pdb session stops inside `beta_function`, inspects $(x, y) = (2, 3)$, steps through validation, and confirms the known result $1/12$.

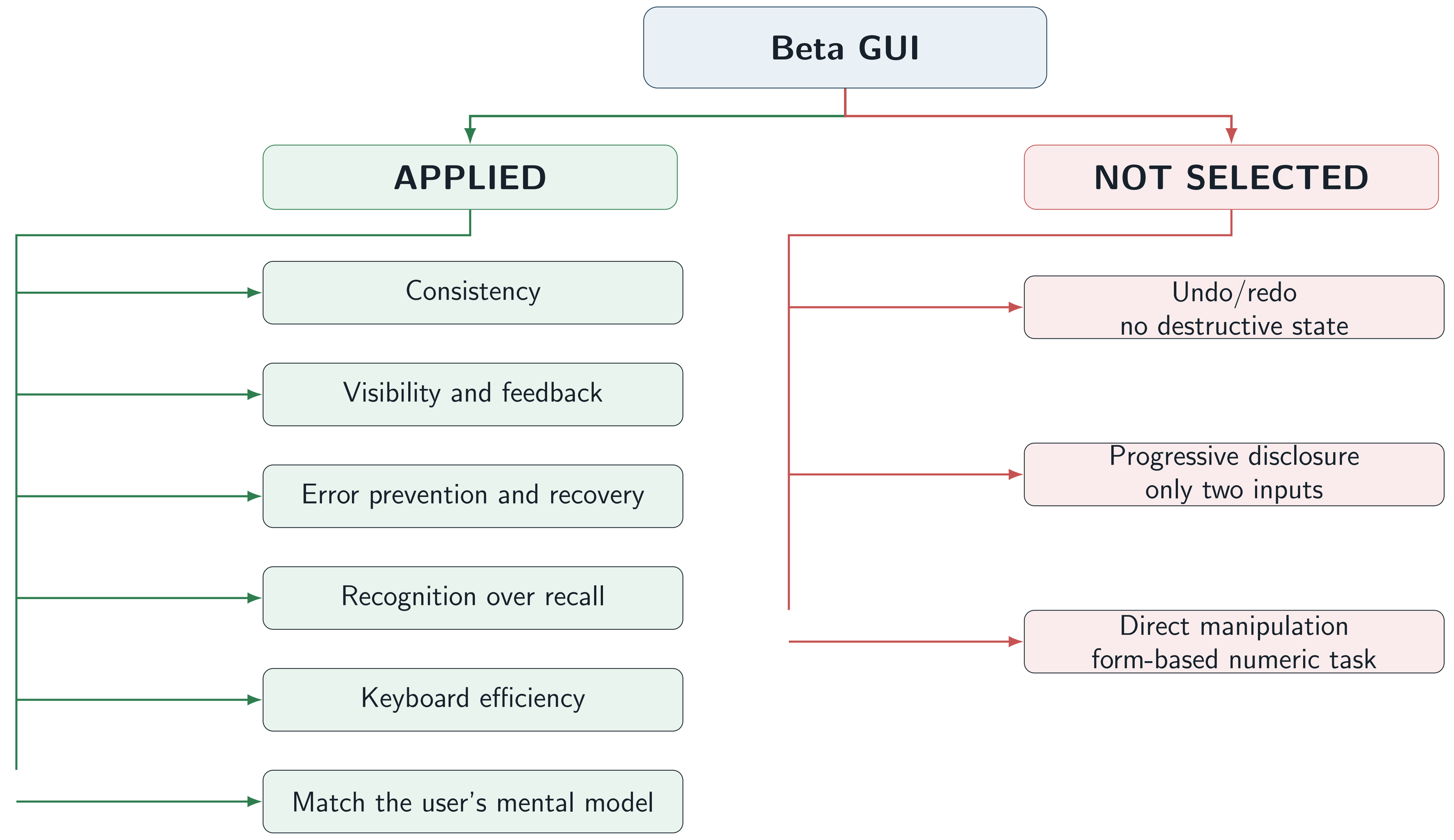
Verified Numerical Contract

External accuracy verification

```
$ .venv/bin/python verify_accuracy.py
External verification only - mpmath, 100 decimal digits
Seed: 6011; generated cases: 500
Inputs: log-uniform from 1e-12 through 1e12
Normal representable results checked: 414
Range-limited reference results excluded: 86
Failures above 1e-10 relative error: 0
Maximum relative error: 1.223e-11
Worst normal-result input: (131117.64197605842, 81.32262207870075)
Large regression B(1e100, 1) relative error: 5.419e-13
Roundable boundary exp(-745): 4.9406564584124654e-324
```

Contract: finite positive inputs produce a representable binary64 result or a specific range error. Twelve displayed digits are formatting, not a universal accuracy claim; subnormal values follow binary64 spacing.

UIDP Decision Mind Map



The mind map records both selected and rejected principles instead of treating every principle as mandatory (Kamthan, 2026d).

Problem 8: PyUnit Test Suite

PyUnit test evidence

```
$ python3 -m unittest -v
test_half_half_equals_pi                      ok
test_known_value_one_one                      ok
test_known_value_two_three                    ok
test_large_asymmetric_identity                ok
test_near_maximum_asymmetric_identity         ok
test_nonfinite_input_is_rejected              ok
test_nonnumeric_input_is_rejected             ok
test_roundable_subnormal_exponential          ok
test_scientific_notation_parser               ok
test_symmetry                                 ok
test_underflow_is_reported                    ok
test_unexpected_arithmetic_error_is_not_hidden ok
test_unroundable_exponential_is_rejected      ok
test_zero_is_rejected                         ok

Ran 14 tests
OK
```

14/14 tests pass. Coverage includes trusted values, symmetry, extreme asymmetric inputs, the roundable subnormal boundary, scientific notation, invalid input, underflow, and propagation of unexpected arithmetic errors. The tests exercise the final numerical core and parser.

Generative AI and CASTROFF. Tool: OpenAI ChatGPT/Codex. Problem 7 prompt: "Review the stable log-beta code for PEP 8, Flake8, pdb, Pylint, SemVer, UIDP, accessibility, and honest range behavior." Problem 8 prompt: "Propose PyUnit regressions for large inputs, subnormal rounding, invalid input, and unexpected exceptions." Context: SOEN 6011 D3; Audience: TA/instructor; Style: concise; Task: review; Role: student engineer; Output: suggestions; Format: poster checklist; Feedback: accept only after execution. Suggestions were revised and verified; unverified claims were rejected.

References. Kamthan, P. (2026a), *SOEN 6011 Project Description.* Kamthan, P. (2026b), *Building, Versioning, and Releasing Software.* Kamthan, P. (2026c), *Introduction to Mental Models.* Kamthan, P. (2026d), *Brainstorming and Mind Mapping.* Van Rossum, G., Warsaw, B., and Coghlan, N. (2001), *PEP 8 – Style Guide for Python Code.* Python Software Foundation (2026), *unittest and pdb Documentation.*

Repository: github.com/HW925/SOEN-6011-D3-Beta-Function